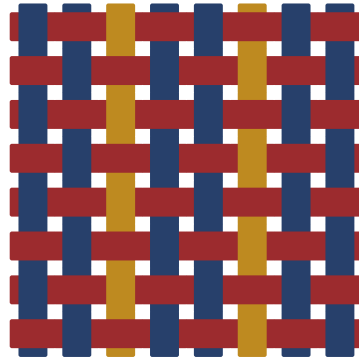




# Information Theory

---

Lecture 2 · from symbols to decodable strings

Oxford B8.4 · 2024--25

fill-in notes from handwritten notes

August 2, 2026

dyed in indigo, madder & weld; ruled in iron-gall. woven on LOOM.

# THE ONE IDEA, AND THE MAP

□□□□

A source code replaces each source symbol by a finite string. Its quality has two sides: **efficiency**, measured by expected length, and **decodability**, tested after codewords are concatenated. The central design problem is to make strings short without losing their boundaries.

## The map so far

| stop | object                                          | question                                              |
|------|-------------------------------------------------|-------------------------------------------------------|
| 1    | $C : \mathcal{X} \rightarrow \mathcal{D}^*$     | Which finite string represents each source symbol?    |
| 2    | $L(C) = \mathbb{E}[\ell(X)]$                    | How many code symbols are used on average?            |
| 3    | $C^* : \mathcal{X}^* \rightarrow \mathcal{D}^*$ | Can concatenated codewords still be decoded uniquely? |

## Conventions

The source alphabet  $\mathcal{X}$  is finite, and  $\mathcal{D}^*$  is the set of all finite-length strings over a  $D$ -ary code alphabet  $\mathcal{D}$ . Most examples use  $\mathcal{D} = \{0, 1\}$ .

# CONTENTS

---

|                                   |          |
|-----------------------------------|----------|
| <b>The one idea, and the map</b>  | <b>1</b> |
| <b>Lecture 2 · Source coding</b>  | <b>3</b> |
| 1 Codes, length, and decodability | 3        |

# LECTURE 2 · SOURCE CODING

Assign strings to symbols, average their lengths under the source law, and then ask the decisive question: does concatenation preserve enough structure to undo the encoding?

## 1 CODES, LENGTH, AND DECODABILITY

□□□□ A codeword names one symbol; an extended code must preserve a whole sequence.

Coding begins with a map from source symbols to finite strings. Short strings save space, but distinct symbols and distinct source sequences must remain distinguishable. This is why expected length and decodability have to be studied together.

### From a source symbol to a codeword

**Definition 1.1** (source code). Let  $X$  be a random variable with finite alphabet  $\mathcal{X}$ . A  $D$ -ary source code is a map

$$C : \mathcal{X} \rightarrow \mathcal{D}^*,$$

where  $\mathcal{D}^*$  is the set of \_\_\_\_\_ over a  $D$ -ary alphabet  $\mathcal{D}$ . The string  $C(x)$  is the **codeword** of  $x$ , and  $\ell(x) = |C(x)|$  is its length.

? The blank is "finite-length strings."

For instance, if  $\mathcal{X} = \{\text{red, blue}\}$  and  $\mathcal{D} = \{0, 1\}$ , one may set  $C(\text{red}) = 00$  and  $C(\text{blue}) = 11$ .

**Definition 1.2** (expected length). If  $X$  has probability mass function  $p$ , the expected length of  $C$  is

$$L(C) = \mathbb{E}[\ell(X)] = \sum_{x \in \mathcal{X}} p(x)\ell(x).$$

**Example 1.3** (an entropy-matched binary code). Let  $\mathcal{X} = \{1, 2, 3, 4\}$ , with probabilities and codewords

| $x$    | 1             | 2             | 3             | 4             |
|--------|---------------|---------------|---------------|---------------|
| $p(x)$ | $\frac{1}{2}$ | $\frac{1}{4}$ | $\frac{1}{8}$ | $\frac{1}{8}$ |
| $C(x)$ | 0             | 10            | 110           | 111           |

The codeword lengths are 1, 2, 3, 3, so

$$L(C) = \frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 + \frac{1}{8} \cdot 3 + \frac{1}{8} \cdot 3 = \text{_____} \text{ bits.}$$

Since  $\ell(x) = -\log_2 p(x)$  for every symbol here,

$$H(X) = \sum_x p(x) \log_2 \frac{1}{p(x)} = \text{_____} \text{ bits} = L(C).$$

? Both blanks are  $\frac{7}{4}$  bits.

**Your turn.** Why does the equality  $L(C) = H(X)$  require no averaging trick in this example?

? Here  $\ell(x) = -\log_2 p(x)$  for every  $x$ , so equality holds term by term.

### Three levels of decodability

■ **Definition 1.4** (*nonsingular code*). A code  $C$  is **nonsingular** if different source symbols have different codewords:

$$x \neq x' \implies C(x) \neq C(x').$$

Equivalently,  $C$  is \_\_\_\_\_.

? The missing word is "injective."

Nonsingularity only protects one symbol at a time. To encode a sequence, extend  $C$  by concatenation:

$$C^* : \mathcal{X}^* \longrightarrow \mathcal{D}^*, \quad C^*(x_1 x_2 \cdots x_n) = C(x_1) C(x_2) \cdots C(x_n).$$

■ **Definition 1.5** (*uniquely decodable code*). A code is **uniquely decodable** if its extension  $C^*$  is nonsingular. Thus every encoded string has at most one source sequence that produced it.

■ **Definition 1.6** (*prefix code*). A code is a **prefix code**, or instantaneous code, if no codeword is a prefix of any other codeword.

Every prefix code is uniquely decodable, and every uniquely decodable code is nonsingular:

$$\{\text{prefix codes}\} \subset \{\text{uniquely decodable codes}\} \subset \{\text{nonsingular codes}\}.$$

The converses need not hold.

■ **Example 1.7** (*uniquely decodable but not prefix*). For  $\mathcal{X} = \{A, B, C, D\}$ , consider

$$C(A) = 0, \quad C(B) = 01, \quad C(C) = 011, \quad C(D) = 111.$$

This is not a prefix code because 0 is a prefix of both 01 and 011. Nevertheless, it is uniquely decodable. Removing an initial 0 from a longer codeword leaves only 1 or 11; comparing these suffixes with 111 swaps  $1 \leftrightarrow 11$  and never produces the empty suffix. Hence no two source sequences have the same concatenation.

**Your turn.** The code  $\{0, 01, 10\}$  is nonsingular but not uniquely decodable. Exhibit two different source sequences with the same encoded string.

?  $AC$  and  $BA$  both encode to 010.